# MODULE-5

Introduction to Approximation Algorithms: Approximation Ratio, 2-approximation algorithm for Vertex Cover problem .Introduction to Randomized Algorithms: Review of Basic Probability, and Polynomial Identity Testing, Randomized Quick Sort. Practical Exposure to basic functionalities of Schwartz-Zippel Lemma

# 1. Introduction to Approximation Algorithms

## 1.1 What is an approximation algorithm?

- An approximation algorithm is an algorithm that finds a **solution close to the optimal solution** for a difficult optimization problem in a **reasonable amount of time**.
- Some optimization problems are difficult to solve exactly, especially when the number of input elements becomes large. Instead of spending a very long time finding the best possible solution, we use an approximation algorithm to obtain a solution that is guaranteed to be reasonably good.

### Simple real-life example

Suppose a **delivery company** must visit several locations while minimizing its travel distance.



- The optimal solution might require finding the shortest possible route among many alternatives.
- Finding the exact shortest route can be computationally difficult for large instances.
- An approximation algorithm finds a good route without examining every possible route.

The approximate route may be slightly longer than the shortest route, but it can be found much faster.

## 1.2 Important Terminology in Approximation Algorithms

| Term | Meaning |
|---|---|
| **Optimization problem** | A problem in which we minimize or maximize an objective. |
| **Feasible solution** | Any solution that satisfies all problem constraints. |
| **Optimal solution** | The best feasible solution. |
| **Approximate solution** | A feasible solution that is close to optimal. |
| **Approximation algorithm** | An algorithm designed to find a good approximate solution. |
| **Approximation ratio** | A measure of how close the approximate solution is to the optimal solution. |

# 1.3 Types of Optimization Problems

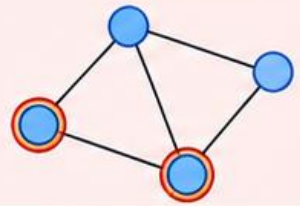
Optimization problems are generally classified into two types.

## 1 Minimization problems

The objective is to find the **smallest possible value**.

**Examples:**

- Minimum Vertex Cover
- Travelling Salesperson Problem (minimum-distance version)
- Minimum Set Cover



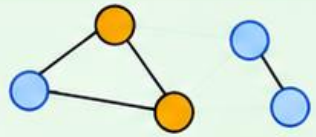
Select minimum number of vertices to cover all edges (Minimum Vertex Cover)

## 2 Maximization problems

The objective is to find the **largest possible value**.

**Examples:**

- Maximum Independent Set
- Maximum Clique
- Maximum coverage problems

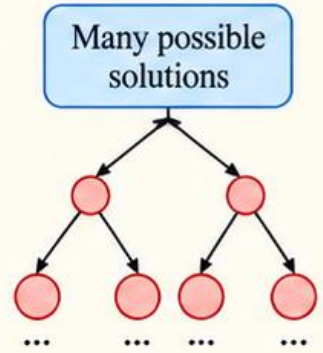


Select maximum number of vertices with no edges between them (Maximum Independent Set)

# 1.4 Why do we need approximation algorithms?

## Problem with exact algorithms

- A problem may have thousands of possible combinations.
- An exact algorithm may need an enormous amount of time to examine those combinations.
- For large instances, finding the optimal solution can be computationally difficult.




Number of combinations grows exponentially

## Advantages of approximation algorithms

1. They provide **efficient solutions** to computationally difficult problems.
2. They **reduce computation time**.
3. They provide **measurable guarantees** on solution quality.
4. They are useful in **scheduling, networking, routing, resource allocation, and logistics**.



**Near-optimal solution in less time**

## 1.5 Exact algorithms versus approximation algorithms

| Exact algorithm | Approximation algorithm |
|---|---|
| Finds the optimal solution. | Finds a feasible solution close to optimal. |
| May require considerable time for difficult problems. | Often runs in polynomial time. |
| Has no solution-quality gap when it completes correctly. | Has a bounded solution-quality gap when a guarantee is proved. |
| Example: exhaustive search for minimum vertex cover. | Example: the 2-approximation algorithm for vertex cover. |

**Important:** An approximation algorithm does not necessarily find the optimal answer. Its advantage is that it can find a good answer efficiently, with a mathematical guarantee.

# 2. Approximation Ratio

## 2.1 Definition

The approximation ratio measures the quality of a solution produced by an approximation algorithm compared with the optimal solution.

**Let:**

- $C$ = value of the solution produced by the algorithm.
- $C^*$ = value of the optimal solution.

### Minimization problem

The approximation ratio for a particular instance is

$$\rho = \frac{C}{C^*}$$

Since the approximate solution cannot be better than the optimal solution,

$$C \geq C^*$$

Therefore,

$$\rho \geq 1$$

### Maximization problem

For a maximization problem, the ratio is commonly written as

$$\rho = \frac{C^*}{C}$$

where $C \leq C^*$, so this ratio is also at least 1.

An algorithm has an approximation guarantee of $\rho$ if its solution satisfies the appropriate bound for every valid input.

# 3. Vertex Cover Problem

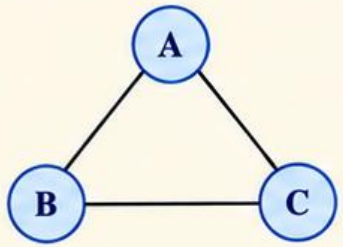
## 3.1 What is a graph?

A graph is a data structure consisting of:

- **Vertices ($V$):** Objects or points.
- **Edges ($E$):** Connections between pairs of vertices.

**Example:**

Consider a graph with three vertices $A$, $B$, $C$ and edges $AB$, $BC$, and $AC$.




$V = \{A, B, C\}$

$E = \{AB, BC, AC\}$

Example graph: $V = \{A, B, C\}$; $E = \{AB, BC, AC\}$

## 3.2 Definition of Vertex Cover

A **vertex cover** of an undirected graph is a set of vertices such that every edge in the graph has at least one endpoint in that set.

**In simple words:** we must select vertices so that every edge is connected to at least one selected vertex.

The objective of the **Minimum Vertex Cover** problem is to find a vertex cover containing the smallest possible number of vertices.

For a graph $G = (V, E)$, find a set $C \subseteq V$ such that

$$\forall (u, v) \in E, \quad u \in C \text{ or } v \in C$$

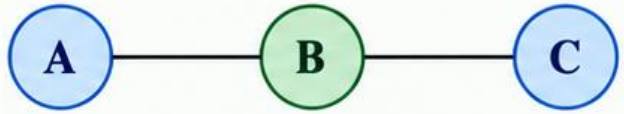
while minimizing

$$|C|$$

Here, $|C|$ represents the number of selected vertices.

## 3.3 Simple example

Consider a graph with vertices $A$, $B$, $C$ and edges $AB$ and $BC$.




Vertex $B$ covers both edges $AB$ and BC.

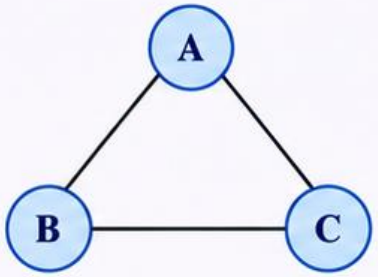
The set $C = \{B\}$ is a vertex cover because:

- Edge $AB$ has endpoint $B$ in the set.
- Edge $BC$ has endpoint $B$ in the set.

Only one vertex is needed. Therefore, the minimum vertex cover has size 1.

## 3.4 Another example: Triangle graph

Consider a triangle with edges $AB$, $BC$, and $AC$.




Possible vertex covers include:

- $\{A, B\}$
- $\{A, C\}$
- $\{B, C\}$

For example, $\{A, B\}$ covers all three edges:

| Edge | Selected endpoint |
|---|---|
| AB | A or B |
| BC | B |
| AC | A |

Selecting just one vertex cannot cover all three edges.

Therefore, $OPT = 2$.

## 3.5 Why is Minimum Vertex Cover difficult?

- Finding the minimum vertex cover in a general graph is an NP-hard optimization problem.
- For large graphs, checking every possible subset of vertices can require exponential time.

**Role of approximation algorithms:**

An approximation algorithm helps us find a vertex cover efficiently without guaranteeing that it is always the smallest possible cover.

# 4. The 2-Approximation Algorithm for Vertex Cover

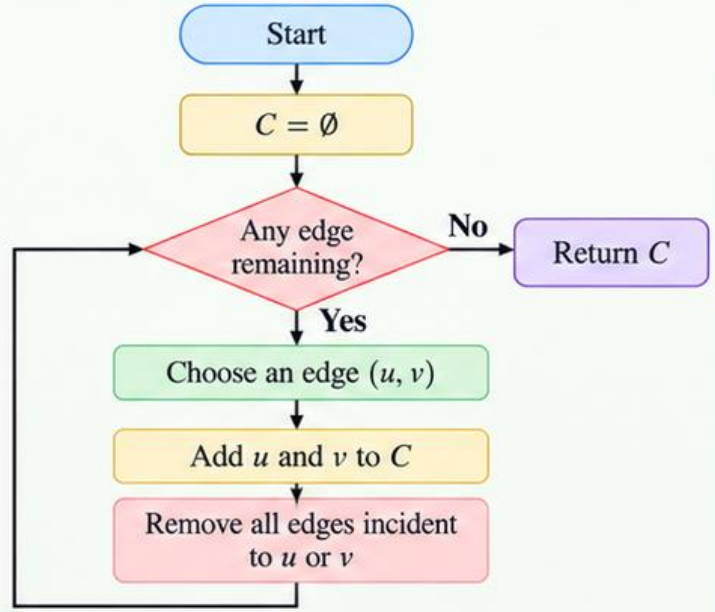
## 4.1 Basic idea

- The 2-approximation algorithm repeatedly selects an edge and includes **both endpoints of that edge** in the vertex cover.
- After selecting the edge, all edges connected to either endpoint are removed from further consideration.
- This process continues until no edges remain.
- The result is a vertex cover whose size is **at most twice the size of the minimum vertex cover**.

## 4.2 Algorithm steps

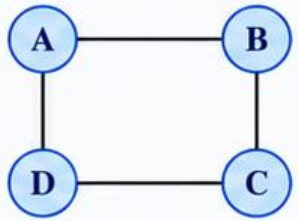
1. Start with an empty vertex cover $C = \emptyset$.
2. While at least one edge remains, choose any edge $(u, v)$.
3. Add both $u$ and $v$ to $C$.
4. Remove every edge incident to $u$ or $v$.
5. Repeat until no edges remain.
6. Return $C$.

## Flowchart of the algorithm



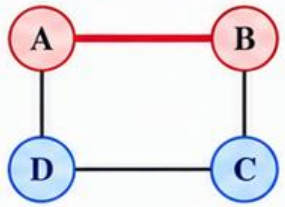

## Example: Applying the algorithm

### Initial graph



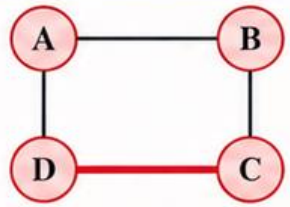

$E = \{AB, BC, CD, DA\}$

### Step 1: Choose edge *AB*



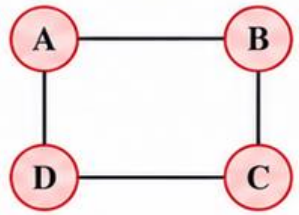

Add $A$ and $B$ to $C$.
Remove edges $AB$, $BC$, $DA$.
Remaining edge: $CD$

### Step 2: Choose edge CD



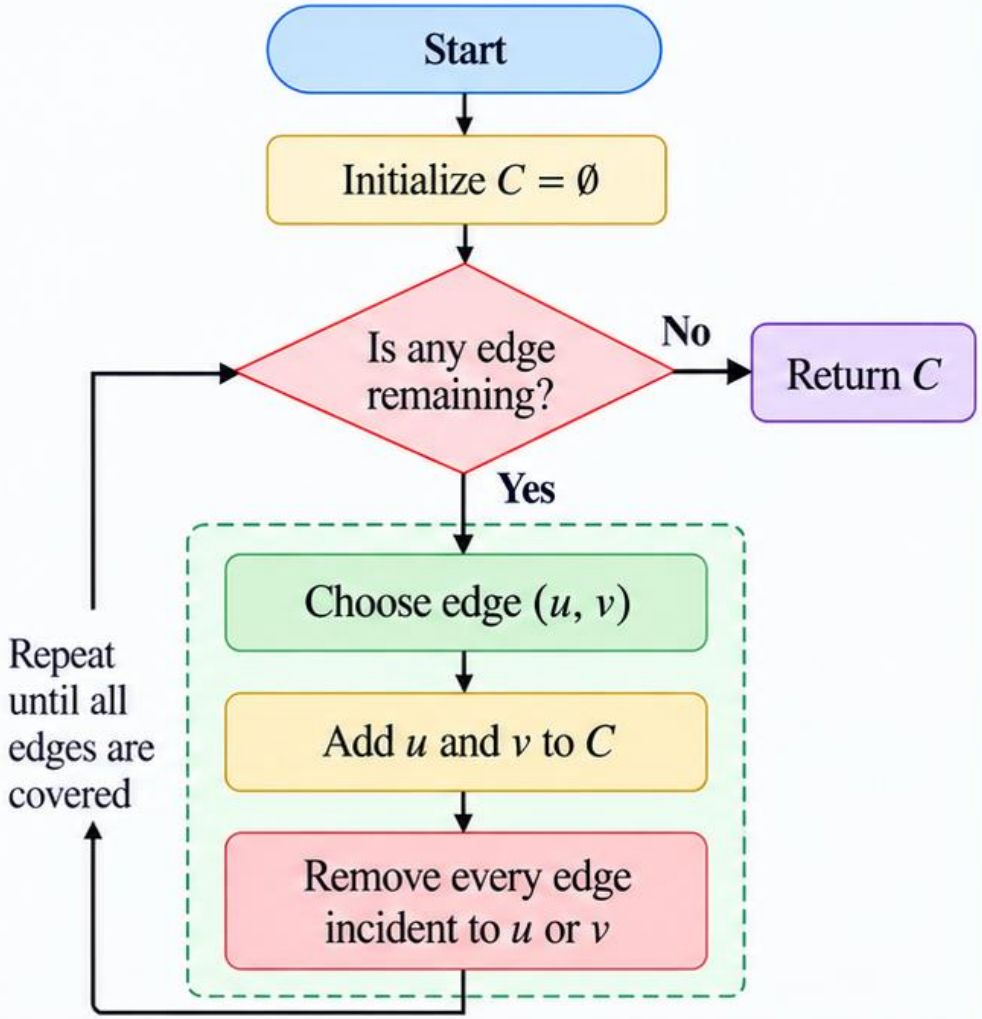

Add $C$ and $D$ to $C$.
Remove edge $CD$.
No edges remain.

### Final vertex cover




$C = \{A, B, C, D\}$

# Flow of the algorithm



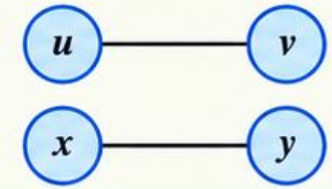

**Repeat the selection and removal steps until all edges have been covered.**

# 4.5 Why is it called a 2-approximation?

**1** Suppose the algorithm selects $k$ edges during its execution. Each selected edge contributes two endpoints to the cover, so the algorithm returns a cover of size

$$|C| = 2k$$

**2** The selected edges are pairwise vertex-disjoint: once an edge is selected, all edges incident to its endpoints are removed. Consequently, no two selected edges share a vertex.




Selected edges are vertex-disjoint

**3** A vertex cover must contain at least one endpoint from every selected edge. Since those edges do not share vertices, at least $k$ distinct vertices are required in any vertex cover.

$$OPT \geq k$$

**4** Combining the two results,

$$|C| = 2k \leq 2OPT$$

**5** Thus,

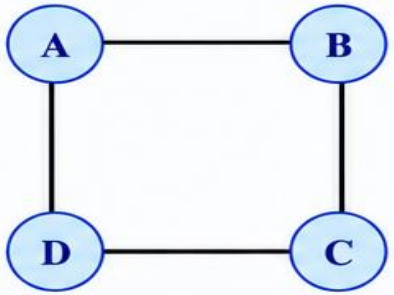
$$|C| \leq 2OPT$$

**6** This proves that the algorithm is a **2-approximation algorithm**.
It guarantees a cover of size no greater than twice the optimal size.

# Example: 2-Approximation Algorithm for Vertex Cover

## 1. Example graph

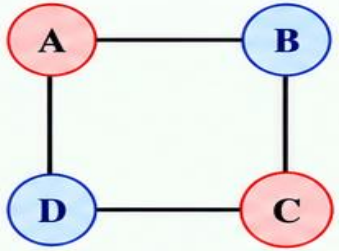
Consider the following graph with 4 vertices and 4 edges.




**Vertices:** $V = \{A, B, C, D\}$
**Edges:** $E = \{AB, BC, CD, DA\}$

## 2. Find the optimal vertex cover

Choose vertices $A$ and $C$.

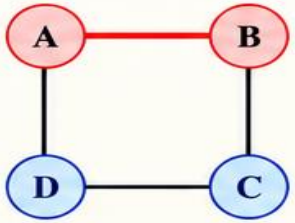



- $A$ covers edges $AB$ and $DA$.
- $C$ covers edges $BC$ and $CD$.
- Therefore, all edges are covered.

$$OPT = \{A, C\}$$
$$|OPT| = 2$$

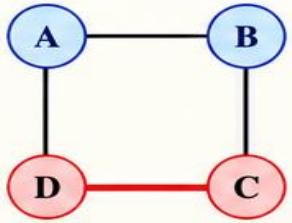
## 3. Apply the 2-approximation algorithm

### Step 1: Select edge $AB$



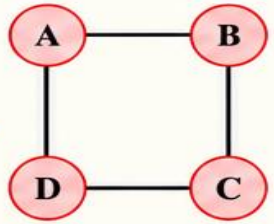

- Choose edge $AB$.
- Add both endpoints $A$ and $B$ to the cover: $C = \{A, B\}$.
- Remove all edges incident to $A$ or $B$ ($AB$, $BC$, $DA$).
- Remaining edge: $CD$.

### Step 2: Select edge $CD$




- Choose edge $CD$.
- Add both endpoints $C$ and $D$ to the cover: $C = \{A, B, C, D\}$.
- Remove edge $CD$.
- No edges remain.

### Step 3: Final vertex cover




**Final vertex cover:**
$C = \{A, B, C, D\}$
$|C| = 4$

## 4. Compare the results

| Solution | Number of vertices |
|---|---|
| Optimal vertex cover (minimum) | 2 |
| Vertex cover from 2-approximation algorithm | 4 |

The approximation ratio is

$$\rho = \frac{|C|}{OPT} = \frac{4}{2} = 2$$

The algorithm selects 4 vertices, while the optimal solution needs only 2.

## 5. Why is it called a 2-approximation?

1. The algorithm selects $k$ edges during its execution. In this example, $k = 2$ (edges $AB$ and $CD$).
2. Each selected edge contributes two endpoints to the cover, so the algorithm returns a cover of size
$$|C| = 2k = 2 \times 2 = 4$$
3. The selected edges are vertex-disjoint. Therefore, any vertex cover must contain at least one endpoint from each selected edge. Hence, $OPT \geq k = 2$
4. Combining the two results, $|C| = 2k \leq 2OPT$
5. Thus, the algorithm guarantees a cover of size no more than twice the optimal size.

### Final result

$$|C| \leq 2OPT$$

This proves that the algorithm is a **2-approximation** algorithm for Vertex Cover.

# 1. Introduction to Randomized Algorithms

## 1.1 What is a randomized algorithm?

A **randomized algorithm** is an algorithm that uses **random choices** while solving a problem.

- Unlike a normal algorithm, which usually follows the same sequence of steps for the same input, a randomized algorithm may make different choices on different runs, even when the input is the same.

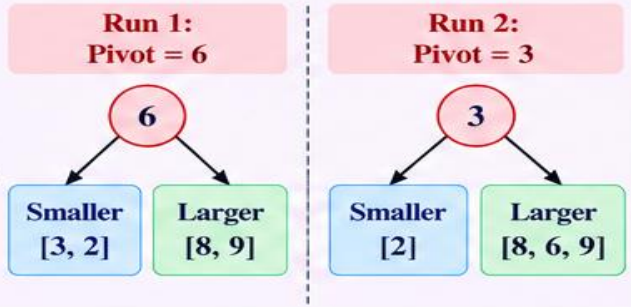
### Simple example

Suppose we want to sort the numbers:

[ 8, 3, 6, 2, 9 ]

A randomized sorting algorithm may randomly choose a pivot, such as 6, to divide the numbers into smaller groups. On another run, it may choose 3 as the pivot.

The choices are different, but the goal remains the same: sort the numbers correctly.




## 1.2 Why do we need randomized algorithms?

**Randomized algorithms are useful because they can:**

1. Reduce the chance of consistently poor performance.
2. Make difficult problems easier to solve efficiently.
3. Simplify algorithm design.
4. Provide good expected running time.
5. Help test whether two large mathematical expressions are equal without expanding them completely.

### Examples of randomized algorithms:

- Randomized Quick Sort
- Randomized primality testing
- Polynomial identity testing

## 1.3 Types of randomized algorithms

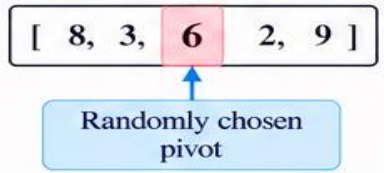
There are two important types of randomized algorithms.

### A. Las Vegas algorithms

- A Las Vegas algorithm always produces the correct answer, but its running time may vary.
- **Example:** Randomized Quick Sort.
- It chooses pivots randomly.
- It always sorts the elements correctly.
- Its running time depends on the pivot choices.

**Example: Randomized Quick Sort**

Randomly choose a pivot, partition the array, and recursively sort.




### B. Monte Carlo algorithms

- A Monte Carlo algorithm has a bounded or predictable running time, but it may produce an incorrect answer with a small probability.
- **Example:** Randomized polynomial identity testing.
- It randomly evaluates a polynomial.
- It can quickly determine whether a polynomial is likely to be zero.
- There is a small probability of error, which can be bounded mathematically.

**Example: Randomized polynomial identity testing**

Choose a random value for the variables and evaluate the polynomial.

If the polynomial is non-zero, it is unlikely to evaluate to 0 at a random point.

| Feature | Las Vegas | Monte Carlo |
|---|---|---|
| Correctness | Always correct | May have a small error probability |
| Running time | Random or variable | Usually has a specified bound |
| Example | Randomized Quick Sort | Randomized polynomial identity testing |

**Remember:** Las Vegas means correct answer with variable running time. **Monte Carlo** means bounded running time with a possible error.

# 2. Review of Basic Probability

**Probability** is important because randomized algorithms use random choices. We need probability to analyze how likely an algorithm is to make a good choice or produce an incorrect result.

## 2.1 Basic terms

| Term | Simple meaning |
|---|---|
| **Random experiment** | An action with an uncertain outcome |
| **Sample space** | The set of all possible outcomes |
| **Event** | A particular outcome or group of outcomes |
| **Probability** | A numerical measure of how likely an event is |
| **Random variable** | A variable whose value depends on a random outcome |

### Example: Tossing a coin

When a fair coin is tossed, the sample space is

$$S = \{H, T\}$$

where $H$ means heads and $T$ means tails.

The probability of heads is

$$P(H) = \frac{1}{2}$$

Similarly,

$$P(T) = \frac{1}{2}$$




## 2.2 Probability formula

When all outcomes are equally likely,

$$P(A) = \frac{\text{Number of favorable outcomes}}{\text{Total number of possible outcomes}}$$

### Example: Probability of an even number on a die

A fair six-sided die is rolled. Find the probability of getting an even number.

The possible outcomes are $S = \{1, 2, 3, 4, 5, 6\}$.

The even numbers are $\{2, 4, 6\}$.

Therefore,

$$P(\text{even}) = \frac{3}{6} = \frac{1}{2}$$



## 2.3 Important probability rules

### Rule 1: Probability of an event and its complement

The complement of an event means that the event does not occur.

$$P(\text{not } A) = 1 - P(A)$$

**Example**

If the probability of success is 0.8, the probability of failure is

$$P(\text{failure}) = 1 - 0.8 = 0.2$$

### Rule 2: Addition rule

For two events $A$ and $B$,

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

If the events cannot occur together, then

$$P(A \cup B) = P(A) + P(B)$$

**Example**

When rolling a die, find the probability of getting 1 or 2.

$$P(1 \text{ or } 2) = \frac{1}{6} + \frac{1}{6} = \frac{1}{3}$$

### Rule 3: Multiplication rule for independent events

Two **events** are **independent** if the occurrence of one does not affect the probability of the other.

$$P(A \cap B) = P(A)P(B)$$

**Example**

Toss a fair coin twice. Find the probability of getting heads both times.

$$P(HH) = \frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$$

### Rule 4: Conditional probability

Conditional probability is the probability of an event occurring when another event is already known to have occurred.

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

provided $P(B) > 0$.

This is useful when a randomized algorithm makes choices based on previous results.

# 2.4 Expected Value and 2.5 Probability of Repeated Independent Failures

## 2.4 Expected value

The **expected value** is the average value we would expect over many repetitions of a random experiment.

**For a discrete random variable $X$,**

$$E[X] = \sum_x xP(X = x)$$

### Example: Fair die

A fair die is rolled. Find its expected value.

**Step 1: List the possible outcomes**

$S = \{1, 2, 3, 4, 5, 6\}$

Each outcome has probability $\frac{1}{6}$.



**Step 2: Apply the formula**

$$E[X] = 1\left(\frac{1}{6}\right) + 2\left(\frac{1}{6}\right) + 3\left(\frac{1}{6}\right) + 4\left(\frac{1}{6}\right) + 5\left(\frac{1}{6}\right) + 6\left(\frac{1}{6}\right)$$

**Step 3: Simplify**

$$E[X] = \frac{1+2+3+4+5+6}{6} = \frac{21}{6} = 3.5$$

**Conclusion:**
The result of a single roll cannot be 3.5, but the average over many rolls approaches 3.5.

## 2.5 Probability of repeated independent failures

**Suppose** a randomized test has an **error probability** of at most $p$ per independent trial.

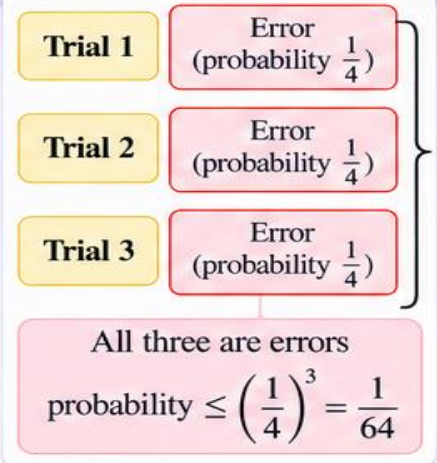
If we repeat the test independently $k$ times and accept an answer only when all trials agree in the required way, the probability of all trials making the same type of error is at most

$$p^k$$

### Example

If $p = \frac{1}{4}$ and we repeat the test three times ($k = 3$), then the probability that all three trials make the same type of error is at most

$$p^k = \left(\frac{1}{4}\right)^3 = \frac{1}{64}$$




### Probability amplification

- Repeating the test independently reduces the error probability.
- This technique is called **probability amplification**.
- It is useful for reducing the error probability of Monte Carlo algorithms.

# 3. Polynomial Identity Testing (PIT)

## 3.1 What is a polynomial?

A **polynomial** is a mathematical expression made up of variables, constants, and non-negative integer powers of variables.

**Examples:**

- $P(x) = x^2 + 3x + 2$
- $Q(x) = x^2 - 4$

A polynomial can contain one variable or several variables.

**Example (two variables):**

$$P(x, y) = x^2 + xy + y^2$$

## 3.2 What is Polynomial Identity Testing?

**Polynomial Identity Testing (PIT)** is the process of checking whether two polynomial expressions are mathematically identical.

- Two expressions are identical if they produce the same polynomial, not merely the same value at one particular input.

**Example:**

$P(x) = x^2 + 2x + 1$ $\qquad$ $Q(x) = (x + 1)^2$

Expanding $Q(x)$ gives $x^2 + 2x + 1$, so the two polynomials are identical.

$$P(x) \equiv Q(x)$$

← The symbol $\equiv$ means identically equal.

## 3.3 Why do we need PIT?

- Some polynomial expressions are extremely large. Expanding and simplifying them completely may require considerable computation.
- PIT provides a way to test equality without necessarily expanding the entire expression.

**Applications include:**

1. Computer algebra systems.
2. Symbolic mathematical computation.
3. Testing algebraic formulas.
4. Verifying calculations in some algorithms.
5. Checking identities involving large polynomials and matrices.

## 3.4 Two types of PIT

### A. Deterministic PIT

- A deterministic algorithm checks polynomial identity using a fixed procedure and guarantees the correct result.
- For general arithmetic circuits, no deterministic polynomial-time PIT algorithm is known in full generality.

Always correct, but may be computationally hard for general cases.

### B. Randomized PIT

- A randomized algorithm chooses random values for the variables and evaluates the expressions at those values.
- If the evaluated values differ, the polynomials are definitely not identical.
- If the evaluated values are equal, the polynomials are likely identical, but there may be a small probability of error.

Fast and simple, with a small probability of error.

## 3.5 Simple example of PIT

Check whether $P(x) = x^2 + 2x + 1$ and $Q(x) = (x + 1)^2$ are identical using random evaluation.

| | |
|---|---|
| **Step 1:** Choose a random value | Suppose $x = 3$. |
| **Step 2:** **Evaluate** $P(x)$ | $P(3) = 3^2 + 2(3) + 1 = 9 + 6 + 1 = 16$ |
| **Step 3:** **Evaluate** $Q(x)$ | $Q(3) = (3 + 1)^2 = 16$ |
| **Step 4:** **Compare the results** | $P(3) = Q(3) = 16$ |

- The result suggests that the expressions are identical. However, one matching evaluation alone does not prove identity.
- A randomized PIT algorithm uses a suitable random sampling method and a mathematical bound on the probability of a false match.

# 4. Randomized Quick Sort

## 4.1 What is Quick Sort?

**Quick Sort** is a sorting algorithm that arranges elements in ascending or descending order using the divide-and-conquer technique.

**It works by:**

1. Choosing a pivot element.
2. Partitioning the array around the pivot.
3. Recursively sorting the elements on either side of the pivot.

In **Randomized Quick Sort**, the pivot is chosen randomly rather than always choosing a fixed position.

## 4.2 How does Randomized Quick Sort work?

Consider the array:

| 8 | 3 | 6 | 2 | 9 |
|---|---|---|---|---|

Suppose the algorithm randomly selects **6** as the **pivot**.

- Elements smaller than 6: [3, 2]
- Elements greater than 6: [8, 9]

The array is now conceptually divided into

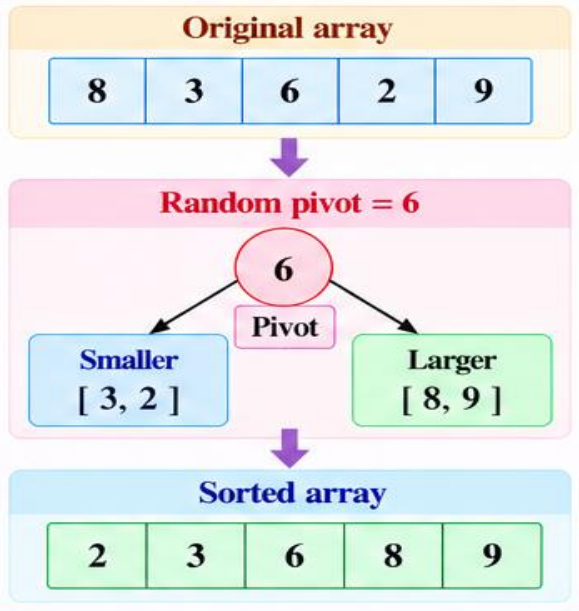
[ 3, 2 ] | 6 | [ 8, 9 ]

Next, sort the left group [3, 2] and the right group [8, 9].

- Left group becomes [2, 3].
- Right group remains [8, 9].

The final sorted array is [ 2, 3, 6, 8, 9 ]

### Visual working of Quick Sort




## 4.3 Algorithm steps

1. If the array contains zero or one element, return it.
2. Randomly choose a pivot.
3. Partition the array into elements smaller than the pivot and elements greater than the pivot. Handle equal elements consistently.
4. Recursively sort the smaller partition.
5. Recursively sort the larger partition.
6. Combine the sorted left partition, pivot, and sorted right partition

### Step 1: Choose a pivot (randomly)

Array: [ 8 3 6 2 9 ]

[ 8, 3 | 6 | 2 9 ]

Randomly chosen pivot = 6

### Step 2: Partition

[ 8, 3 | 6 | 2 9 ]

Elements < 6: [ 3, 2 ]

Elements > 6: [ 8, 9 ]

Conceptual division:

[ 3, 2 ] | 6 | [ 8, 9 ]

### Step 3: Recursively sort both parts

Sort left part [3, 2] → [2, 3]

| 2 | 3 |
|---|---|

Sort right part [8, 9] → [8, 9]

| 8 | 9 |
|---|---|

### Step 4: Combine results

Combine sorted left part, pivot and sorted right part.

| 2 | 3 | 6 | 8 | 9 |
|---|---|---|---|---|

Final sorted array

# 4.5 Time complexity of Randomized Quick Sort

The running time of **Randomized Quick Sort** depends on how the array is partitioned, which in turn depends on the randomly chosen pivot.

| Case | Time complexity | Explanation |
|---|---|---|
| **Best case** | $O(n \log n)$ | Partitions are reasonably balanced. |
| **Expected case** | $O(n \log n)$ | Random pivot choices give expected balanced progress. |
| **Worst case** | $O(n^2)$ | Partitions repeatedly contain almost all elements on one side. |

- The **expected time complexity** is $O(n \log n)$, assuming the usual randomized pivot selection and standard partitioning.
- The **worst case** is still $O(n^2)$, but random pivot selection helps avoid consistently bad partitions for many input patterns.

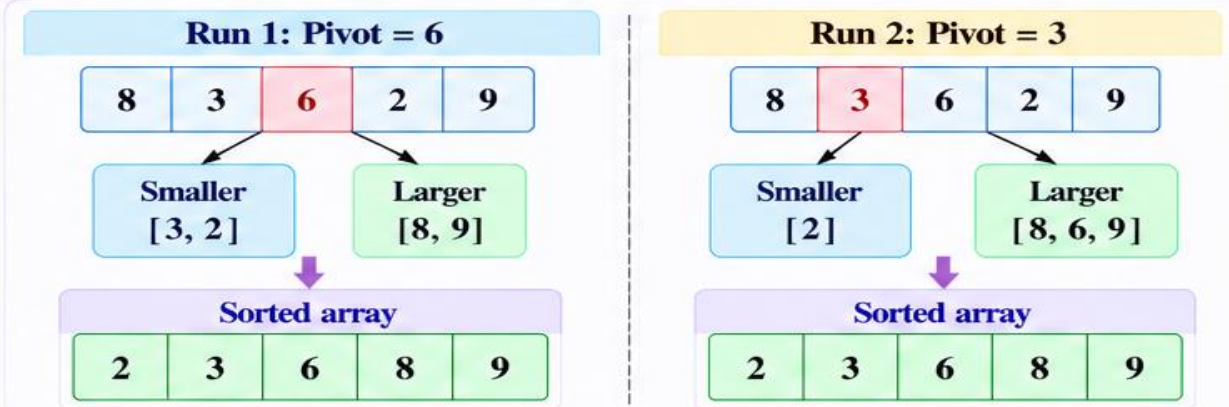
# 4.6 Why is Randomized Quick Sort a Las Vegas algorithm?

- Randomized Quick Sort always sorts the array **correctly**, regardless of which pivots are selected.
- Only its **running time changes** depending on the random choices.
- Therefore, it is a **Las Vegas randomized algorithm**.

### Remember: Las Vegas algorithm

- Always produces the correct answer.
- Its running time may vary depending on the random choices.

### Different runs with different pivots (same correct result)




The pivot may be different in different runs, but the final sorted array is always correct.

# Schwartz – Zippel Lemma

## 1. What is the Schwartz–Zippel Lemma?

The **Schwartz–Zippel Lemma** tells us that if a polynomial is not identically zero, the probability of getting zero after substituting randomly chosen values into it is limited.

### In simple words:

If a polynomial is not really zero, randomly choosing values for its variables is unlikely to make its result zero, provided we choose from a sufficiently large set.

### Why is this useful?

We can use random values to check whether two complicated polynomial expressions are identical without expanding and comparing every term.

## 2. First, understand a simple polynomial

Consider $P(x) = x - 5$. Let's substitute different values of $x$.

| Value of $x$ | Calculation | Result |
|---|---|---|
| **1** | $1 - 5$ | $-4$ |
| **2** | $2 - 5$ | $-3$ |
| **3** | $3 - 5$ | $-2$ |
| **5** | $5 - 5$ | **0** |
| **8** | $8 - 5$ | 3 |

Notice that the polynomial gives zero only when $x = 5$.

Suppose we randomly choose $x$ from the set

$$S = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$$

There are 10 possible values, but only one makes the polynomial zero.

Therefore,

$$P(P(x) = 0) = \frac{1}{10}$$

More precisely,

$$\Pr[P(x) = 0] = \frac{1}{10}$$

So, the probability of getting zero is only **10%**.

## 3. Statement of the Schwartz–Zippel Lemma

Let $P(x_1, x_2, \ldots, x_n)$ be a **non-zero polynomial** with total degree $d$.

Choose each variable **independently and uniformly** from a finite set $S$.

Then,

$$\Pr[P = 0] \leq \frac{d}{|S|}$$

### Meaning of the symbols

| Symbol | Meaning |
|---|---|
| $P$ | The polynomial being tested |
| $d$ | The total degree of the polynomial |
| $S$ | The set from which random values are selected |
| $\lvert S \rvert$ | The number of elements in the set |
| $\Pr[P = 0]$ | Probability that the polynomial evaluates to zero |

**Remember:** The lemma gives an **upper bound** on the probability, not necessarily the exact probability.

# Examples of the Schwartz–Zippel Lemma

## 4. Example 1: A simple linear polynomial

Consider $P(x) = x - 5$.

**1 Find the degree.**

The highest power of $x$ is 1.

Therefore, $d = \mathbf{1}$.

**2 Choose a set of random values.**

Let $S = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$.

Therefore, $|S| = 10$.

**3 Apply the Schwartz–Zippel formula.**

$$\Pr[P(x) = 0] \leq \frac{d}{|S|} = \frac{1}{10}.$$

$$\Pr[P(x) = 0] \leq \mathbf{10\%}.$$

**Explanation**

Only one value, $x = 5$, makes the polynomial zero.

Therefore, the actual probability is $\frac{1}{10}$ (**10%**), which satisfies the bound.

| Value of $x$ | $P(x) = x - 5$ |
|---|---|
| **1** | $-4$ |
| **2** | $-3$ |
| **3** | $-2$ |
| **5** | **0** |
| **8** | 3 |

## 5. Example 2: A quadratic polynomial

Consider $P(x) = x^2 - 4$.

**1 Find the degree.**

The highest power is 2, so $d = \mathbf{2}$.

**2 Select a finite set.**

Let $S = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

Here, $|S| = 10$.

**3 Find when the polynomial becomes zero.**

$$x^2 - 4 = 0$$
$$x^2 = 4$$
$$x = 2 \quad \text{or} \quad x = -2$$

Only $x = 2$ belongs to the selected set $S$, so exactly one of the ten values makes the polynomial zero.

**4 Calculate the actual probability.**

$$\Pr[P(x) = 0] = \frac{1}{10} = \mathbf{10\%}.$$

**5 Apply the Schwartz–Zippel bound.**

$$\Pr[P(x) = 0] \leq \frac{d}{|S|} = \frac{2}{10}.$$

$$\Pr[P(x) = 0] \leq \mathbf{20\%}.$$

**Explanation**

- The actual probability is **10%**, while the upper bound is **20%**.
- The lemma guarantees that the probability cannot exceed the bound; it does not say that the probability must equal 20%.

# 6. Why is the Schwartz–Zippel Lemma useful in Polynomial Identity Testing?

## 6.1 Checking polynomial identity

Suppose we want to check whether two polynomials are identical.

$P(x) = x^2 + 2x + 1$ and $Q(x) = (x+1)^2$

We want to determine whether $P(x) \equiv Q(x)$

- Instead of expanding and comparing complicated expressions, we can evaluate both at randomly selected values.
- The important trick is to create a difference polynomial:

$$R(x) = P(x) - Q(x)$$

- If the two polynomials are identical, then $R(x) \equiv 0$.
- If they are not identical, $R(x)$ is a **non-zero polynomial**.

## 6.2 Step-by-step example

| | | |
|---|---|---|
| 1 | **Choose a random value.** | Suppose we randomly choose $x = 3$. |
| 2 | **Evaluate** $P(x)$. | $P(3) = 3^2 + 2(3) + 1 = 9 + 6 + 1 =$ **16** |
| 3 | **Evaluate** $Q(x)$. | $Q(3) = (3+1)^2 =$ **16** |
| 4 | **Compare the results.** | $P(3) = Q(3) = 16$ |

The expressions match at this random value, so they pass this evaluation test.

## 6.3 Why one matching result is not enough?

- One matching result does not prove that the polynomials are identical.
- Two different polynomials may happen to have the same value at a particular input.

**Example: Two different polynomials with the same value at $x = 1$**

$P(x) = x^2$ and $Q(x) = x$

$P(1) = 1$ and $Q(1) = 1$

They are not identical polynomials, but they give the same value at $x = 1$.

## 6.4 Using the Schwartz–Zippel Lemma

The Schwartz–Zippel Lemma tells us how likely such a false match can be.

Suppose $R(x) = P(x) - Q(x)$ is a **non-zero polynomial** of **degree at most 2**, and the values are selected from a set of 100 elements.

**Apply the lemma:**

$$\Pr[R(x) = 0] \leq \frac{d}{|S|} = \frac{2}{100}$$

**False-positive probability $\leq$ 2%.**

**Here:**

- $d = 2$ (degree of $R(x)$)
- $|S| = 100$ (size of the selected set)

## 6.5 Conclusion

- A random evaluation can test polynomial identity efficiently without expanding the expressions.
- The probability of a false positive (incorrectly concluding that the polynomials are identical) is at most 2% in this example.
- By using a sufficiently large set and repeating the test, the error probability can be made even smaller.

# 9. Advantages and Limitations of the Schwartz–Zippel Lemma

## Advantages

- Helps **test polynomial identities efficiently**.
- **Avoids** the need to **expand very large expressions**.
- Provides a **mathematical upper bound** on error probability.
- **Repeated testing** can **reduce the error probability**.

## Limitations

- A single evaluation can give a **false positive**.
- The error bound depends on the **polynomial degree** and the **sampling set size**.
- The variables must be **sampled according to the lemma's assumptions**.
- A **larger degree** may require a **larger sampling set** to achieve the same guarantee.

# 10. Quick Revision for Examinations

## Remember these five points:

1. The Schwartz–Zippel Lemma applies to a **non-zero polynomial**.
2. $d$ represents the **total degree**, and $|S|$ represents the **size of the sampling set**.
3. The probability that the polynomial evaluates to zero is bounded by the **degree divided by the sampling-set size**.
4. In polynomial identity testing, we test $R = P - Q$.
5. Independent repetitions can **reduce the probability** of a false positive.

## Most important formula

$$\Pr[P = 0] \leq \frac{d}{|S|}$$

**Meaning:**

- $P$ : the polynomial being tested (non-zero polynomial)
- $d$ : total degree of the polynomial
- $S$ : the set from which random values are selected
- $|S|$ : number of elements in the set
- $\Pr[P = 0]$ : probability that the polynomial evaluates to zero